JAVA LAB WEEK 12

Title: Multithreading – Thread Priorities, Synchronization and Creating Multiple Threads (Seat Reservation System)

1. Objective / Aim

To write and execute a Java program to implement a seat reservation system using multithreading. The program creates multiple person threads to book seats and uses the synchronized keyword to ensure that only one thread can access the reservation method at a time.

2. System Requirements

Hardware Requirements:

- Computer or laptop
- Minimum 4 GB RAM
- Keyboard and monitor

Software Requirements:

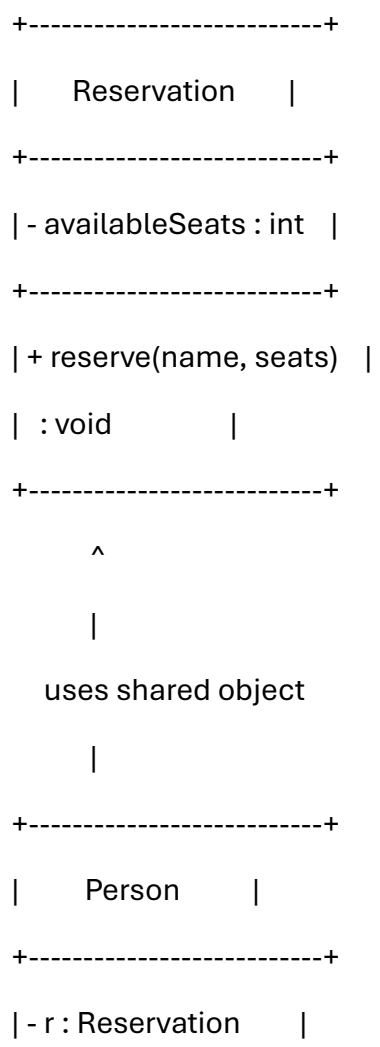
- Operating System: Windows, Linux, or macOS
- Java Development Kit (JDK): JDK 8 or above
- IDE: Eclipse, IntelliJ IDEA, NetBeans, or Visual Studio Code
- Command Prompt or Terminal

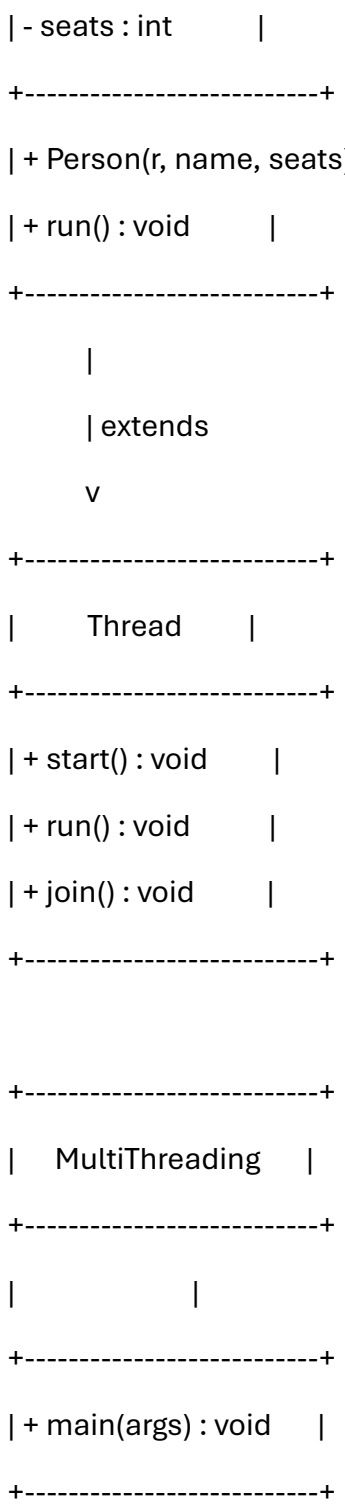
3. Algorithm / Logic Description

1. Start the program.
2. Create a class named Reservation.
3. Declare an integer variable availableSeats and initialize it to 10.
4. Define a synchronized method named reserve() that accepts the person's name and requested seats.
5. Display the person's name, available seats, and requested seats.
6. Check whether the requested seats are available.
7. If seats are available, reserve the seats and reduce the available seat count.
8. Otherwise, display the message that the requested seats are not available.
9. Create a class named Person that extends the Thread class.

10. Declare a Reservation reference and an integer variable to store the requested seats.
11. Create a constructor to initialize the reservation object, person name, and requested seats.
12. Override the run() method to call the synchronized reserve() method.
13. In the main class, create one shared Reservation object.
14. Create three person thread objects with different seat requirements.
15. Start the threads using the start() method.
16. Use the join() method to ensure that each person completes booking before the next person starts.
17. Display the reservation details.
18. Stop the program.

4. UML Class Diagram





Description:

- Reservation is a class that maintains the available seats and provides the reserve() method.
- Person is a thread class that extends the built-in Thread class.
- availableSeats stores the number of remaining seats.

- r stores the shared reference to the Reservation object.
- seats stores the number of seats requested by a person.
- reserve() is a synchronized method that allows only one thread at a time to access the reservation operation.
- run() performs the reservation operation when the thread starts.
- start() starts the execution of a thread.
- join() makes the main thread wait until the specified thread completes.
- MultiThreading is the main class that creates and starts the person threads.

5. Source Code

```
class Reservation {

  int availableSeats = 10;

  synchronized void reserve(String name, int seats) {

    System.out.println(name + " entered.");
    System.out.println("Available seats: " + availableSeats +
                " Requested seats: " + seats);

    if (availableSeats >= seats) {
      System.out.println("Seat Available. Reserve now :-)");
      availableSeats = availableSeats - seats;
      System.out.println(seats + " seats reserved.");
    } else {
      System.out.println("Requested seats not available :-)");
    }

    System.out.println(name + " leaving.");
```

```
        System.out.println("-------------------------------------------");
    }
}

class Person extends Thread {

    Reservation r;
    int seats;

    Person(Reservation r, String name, int seats) {
        this.r = r;
        this.seats = seats;
        setName(name);
    }

    public void run() {
        r.reserve(getName(), seats);
    }
}

public class MultiThreading {

    public static void main(String[] args) throws InterruptedException {

        Reservation r = new Reservation();

        Person p1 = new Person(r, "Person-1", 5);
        Person p2 = new Person(r, "Person-2", 2);
```

```
        Person p3 = new Person(r, "Person-3", 4);

        p1.start();
        p1.join();

        p2.start();
        p2.join();

        p3.start();
        p3.join();
    }
}
```

6. Compilation and Execution Commands

Step 1: Save the program

Save the Java program with the filename MultiThreading.java.

Step 2: Open Command Prompt or Terminal

Navigate to the folder where the Java program is saved.

Step 3: Compile the program

javac MultiThreading.java

Step 4: Execute the program

java MultiThreading

7. Output Screen

Sample Output:

Person-1 entered.

Available seats: 10 Requested seats: 5

Seat Available. Reserve now :-)

5 seats reserved.

Person-1 leaving.

Person-2 entered.

Available seats: 5 Requested seats: 2

Seat Available. Reserve now :-)

2 seats reserved.

Person-2 leaving.

Person-3 entered.

Available seats: 3 Requested seats: 4

Requested seats not available :-)

Person-3 leaving.

Explanation of Output:

Initially, the reservation system has 10 available seats.

Person-1:

- Requested seats = 5
- Available seats = 10
- Booking is successful.
- Remaining seats = 10 - 5 = 5.

Person-2:

- Requested seats = 2
- Available seats = 5
- Booking is successful.
- Remaining seats = 5 - 2 = 3.

Person-3:

- Requested seats = 4
- Available seats = 3
- Booking is unsuccessful because only 3 seats are available.

Therefore, the final available seats are 3.

Important Concepts Used:

- **Multithreading:** Allows multiple threads to execute within a program.
- **Thread Class:** Used to create individual person threads.
- **Synchronization:** The synchronized keyword allows only one thread at a time to execute the reservation method on the shared object.
- **start():** Starts a new thread.
- **run():** Contains the task performed by the thread.
- **join():** Makes the main thread wait until the specified thread completes.

Note: The problem statement mentions 100 initial seats, but the expected output starts with 10 seats. Therefore, this program uses 10 initial seats to match the given expected output. The program uses join() to maintain the same booking order as the expected output. Thread priorities are not explicitly used because they are not required to produce this reservation output.

8. Result

Thus, the Java program to implement a seat reservation system using multithreading and synchronization was successfully implemented and executed. The experiment demonstrates thread creation using the Thread class, synchronized methods, shared resource management, and thread control using start() and join() methods. It ensures that only one person thread can access the reservation method at a time.